

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Observation Task 5 (Part A)

Date: 21-09-2026

Student Name: M. Naga Doleswari Aruna Devi

Roll No.: A24126552267

PART A — Q5

Differentiating JavaScript Functions and Classes, and Creating Multiple Objects Using a Class

1. AIM

To differentiate between JavaScript functions and classes, and to demonstrate, with a working program, how a class can be used to create multiple objects that share common properties and methods.

2. OBJECTIVE

- To understand a regular JavaScript function and how it can be used as a constructor function.
- To understand the class syntax (ES6) as syntactic sugar over JavaScript's prototype-based inheritance.
- To compare functions and classes on definition, invocation, hoisting, strict mode and constructor rules.
- To create multiple objects from the same class, each with its own property values but sharing the same methods.
- To verify, through code, how a class enforces object creation only through the 'new' keyword.

3. THEORY

A JavaScript function is a general-purpose, reusable block of code. It can be called directly (`greet()`), used as a constructor with `new` (`new Student()`), or used as a method, a callback or an IIFE. Because of this flexibility, the value of this inside a normal function depends on how it is called, and calling a constructor function without `new` does not raise an error — it silently attaches properties to the wrong object (the global object, or undefined in strict mode).

A JavaScript class (introduced in ES6) is a template used specifically to create objects with a shared shape. It is defined with the `class` keyword, contains a special `constructor()` method that runs once per object to set initial property values, and any other methods written inside the class body are placed on the class's prototype and shared by every object created from it — they are not copied into each object individually. A class can only be instantiated with `new`; calling it as a plain function (`StudentClass()`) throws a `TypeError`. Class bodies also always execute in strict mode and class declarations are not hoisted the way function declarations are.

The key difference for object creation is this: a function only becomes a way of making objects when it is deliberately used as a constructor (by convention, given a capitalised name and called with `new`), whereas a class exists only to serve as a blueprint for objects. In both cases, methods defined on the prototype (via `Function.prototype.method = ...` for a function, or directly inside a class body) are stored once in memory and shared by every instance, while each instance keeps its own independent copy of the properties set inside the constructor.

3.1. Functions vs Classes — Comparison

Aspect	Function (used as constructor)	Class
Keyword	<code>function</code>	<code>class</code>
Purpose	General-purpose; can also be a constructor by convention	Dedicated blueprint for creating objects
Invocation	Can be called with or without 'new'	Must always be called with 'new'
Calling without 'new'	No error; 'this' binds incorrectly (global object / undefined)	Throws: 'Class constructor ... cannot be invoked without new'
Hoisting	Function declarations are hoisted (usable before definition)	Class declarations are not initialised until evaluated (temporal dead zone)

Aspect	Function (used as constructor)	Class
Strict mode	Not strict unless declared	Body always runs in strict mode automatically
Methods	Added manually via <code>Function.prototype.method = ...</code>	Written directly inside the class body; auto-added to the prototype
Inheritance	Manual prototype chaining (<code>Object.create</code> , <code>.call()</code>)	Built-in 'extends' and 'super' keywords
Re-declaration	Can be re-declared / reassigned	Class name in the same scope cannot be re-declared

How a class produces multiple objects: every time `new StudentClass(name, branch)` is executed, JavaScript creates a brand-new object, runs the `constructor()` with that new object as `this`, and links the object's internal prototype to `StudentClass.prototype`. Each resulting object therefore has its own independent name and branch values, while all of them share the single `display()` method that lives on the prototype — the method is not duplicated for every object, which saves memory as more objects are created.

4. ALGORITHM / PROCEDURE

- Create a file named `app.js`.
- Define a constructor function `StudentFunction(name, branch)` that sets `this.name` and `this.branch`, and add a `display()` method to `StudentFunction.prototype`.
- Define an equivalent class `StudentClass` with a `constructor(name, branch)` and a `display()` method written directly inside the class body.
- Create two objects using `'new StudentFunction(...)'` and call `display()` on each.
- Create three objects using `'new StudentClass(...)'` and call `display()` on each, to show that a class can produce any number of independent objects that share the same method.
- Use `typeof` on both `StudentFunction` and `StudentClass` to show both are technically of type `'function'`.
- Call `StudentClass(...)` without `'new'` inside a `try/catch` block and print the resulting error, to show that a class enforces the use of `'new'`.
- Call `StudentFunction(...)` without `'new'` to show that, unlike a class, no error is raised, and observe that `'this'` inside the call is bound incorrectly.
- Save the file and run it using: `node app.js`
- Record the console output and compare it against the expected behaviour.

5. PROGRAM

`app.js`

```
// Comparing JavaScript Functions and Classes

// 1. Object creation using a constructor FUNCTION
function StudentFunction(name, branch) {
    this.name = name;
    this.branch = branch;
}
StudentFunction.prototype.display = function () {
    console.log(this.name + " - " + this.branch);
};

// 2. Object creation using a CLASS
class StudentClass {
    constructor(name, branch) {
        this.name = name;
        this.branch = branch;
    }
    display() {
        console.log(this.name + " - " + this.branch);
    }
}
console.log("--- Using constructor function ---");
```

```

const f1 = new StudentFunction("Arun", "CSE");
const f2 = new StudentFunction("Priya", "AIML");
f1.display();
f2.display();

console.log("\n--- Using class ---");
const c1 = new StudentClass("Rahul", "ECE");
const c2 = new StudentClass("Sita", "CSE");
const c3 = new StudentClass("Kiran", "AIML");
c1.display();
c2.display();
c3.display();

console.log("\n--- typeof check ---");
console.log("typeof StudentFunction:", typeof StudentFunction);
console.log("typeof StudentClass:", typeof StudentClass);

console.log("\n--- Calling class without 'new' ---");
try {
  StudentClass("Test", "IT");
} catch (err) {
  console.log("Error:", err.message);
}

console.log("\n--- Calling function without 'new' (no error, but 'this' is wrong) ---");
StudentFunction("Test", "IT");
Run command: node app.js

```

6. CODE EXPLANATION

Tag / Statement	Description
function StudentFunction(name, branch) {...}	A regular function used as a constructor; 'this' refers to the new object only when called with 'new'.
StudentFunction.prototype.display = ...	Manually adds a shared method to the function's prototype so every instance can use it without copying it.
class StudentClass { constructor(...) {...} display() {...} }	Defines a class with a constructor (runs per object) and a display() method (added automatically to the class's prototype and shared by all instances).
new StudentFunction(...) / new StudentClass(...)	Creates a new object, runs the constructor/function with 'this' bound to that object, and returns it.
typeof StudentFunction / typeof StudentClass	Both evaluate to 'function', showing that a class is implemented on top of JavaScript's function/prototype system.
try { StudentClass(...) } catch (err) {...}	Calling a class without 'new' throws a TypeError; the try/catch captures and prints that error message.
StudentFunction('Test', 'IT') (no 'new')	Demonstrates that, unlike a class, calling a constructor function without 'new' does not throw — it silently misassigns 'this', a common source of bugs.

7. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Status
TC-01	Create f1, f2 with 'new StudentFunction(...)' and call display()	Arun - CSE and Priya - AIML printed, one line each	Pass
TC-02	Create c1, c2, c3 with 'new StudentClass(...)' and call display()	Rahul - ECE, Sita - CSE and Kiran - AIML printed; three independent objects share one display() method	Pass
TC-03	typeof StudentFunction and typeof StudentClass	Both print 'function'	Pass

Test Case	Action	Expected Result	Status
TC-04	Call StudentClass('Test','IT') without 'new'	TypeError thrown: "Class constructor StudentClass cannot be invoked without 'new'"	Pass
TC-05	Call StudentFunction('Test','IT') without 'new'	No error is thrown; the function runs but 'this' does not refer to a Student object	Pass

8. OUTPUT

Actual console output produced by running the program above with node app.js:

```
$ node app.js
--- Using constructor function ---
Arun - CSE
Priya - AIML

--- Using class ---
Rahul - ECE
Sita - CSE
Kiran - AIML

--- typeof check ---
typeof StudentFunction: function
typeof StudentClass: function

--- Calling class without 'new' ---
Error: Class constructor StudentClass cannot be invoked without 'new'

--- Calling function without 'new' (no error, but 'this' is wrong) ---
```

Fig. 8.1 — Console output comparing constructor-function and class object creation

9. RESULT

Thus, JavaScript functions and classes were studied and compared. It was observed that a function only creates objects when deliberately used as a constructor with 'new', whereas a class is a dedicated blueprint that always requires 'new' and throws an error otherwise. A class (StudentClass) was used to create three independent objects (Rahul, Sita, Kiran), each holding its own name and branch while sharing a single display() method defined on the prototype, confirming that a class allows multiple objects to be created efficiently with common properties and methods. The behaviour was verified through command-line testing, matching the expected results shown above.

PART A — Q6

Express.js Middleware for Logging Incoming Requests

1. AIM

To explain the concept of middleware in Express.js, describe how it works within the request-response cycle, and develop an example of logging middleware that records the HTTP method and URL of every incoming request.

2. OBJECTIVE

- To understand what middleware is in Express.js and its function signature (req, res, next).
- To understand how middleware fits into the request-response cycle and the role of the next() function.
- To understand the difference between application-level middleware and route handlers.
- To develop a logging middleware that records the method and URL of every incoming request.
- To verify, through testing, that the middleware executes before the route handler for every route.

3. THEORY

Middleware in Express.js is a function that has access to the request object (req), the response object (res), and a third argument, next, which is a function used to pass control to the next middleware or route handler in the pipeline. Its general signature is function(req, res, next) { ... }.

A middleware function can perform any of the following: execute code such as logging, parsing or authentication; modify the req or res objects; end the request-response cycle by sending a response; or call next() to pass control onward. Middleware is registered using app.use() (applied to every request) or attached to a specific route as an extra argument before the route handler.

Middleware in the request-response cycle: When a request arrives at an Express server, it passes through the registered middleware functions in the exact order in which they were defined, forming a pipeline: request → middleware 1 → middleware 2 → ... → matching route handler → response. Each middleware must either call next() to hand control to the next function in the chain, or send a response itself (using res.send(), res.json(), res.status().send(), etc.), which ends the cycle at that point. If a middleware neither calls next() nor sends a response, the request hangs and the client never receives a reply. Because middleware registered with app.use() runs before the routes defined after it, it is commonly placed at the top of the file so that it runs for every incoming request, regardless of which route is ultimately matched.

A logging middleware is a simple example of this pattern: it does not send a response itself; it only records information about the request (such as the HTTP method and URL) and then calls next() so that the actual route handler can process the request and send the response.

4. ALGORITHM / PROCEDURE

1. Create a project folder and a file named app.js.
2. Import Express using require("express") and create the application object with express().
3. Define a middleware function using app.use() that runs for every incoming request, before any route is declared.
4. Inside the middleware, read req.method and req.url and print them to the console using console.log().
5. Call next() at the end of the middleware so that control passes on to the matching route handler.
6. Define GET routes for "/", "/student" and "/about", each sending a simple text response using res.send().
7. Start the server on port 3000 using app.listen() and display a confirmation message on the console.
8. Run the program using the command: node app.js
9. Open each route in the browser and observe the method and URL logged on the terminal for every request.
10. Test the middleware with different routes and record the observations against the expected results.

5. PROGRAM

middleware.js

```
const express = require("express");
const app = express();
```

```
// Middleware
app.use((req, res, next) => {
  console.log("Request Method:", req.method);
  console.log("Request URL:", req.url);
  next();
});

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to Middleware Demo");
});

// Student route
app.get("/student", (req, res) => {
  res.send("Student details displayed");
});

// About route
app.get("/about", (req, res) => {
  res.send("This is the About page");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
Run command: node middleware.js
```

6. CODE EXPLANATION

Tag / Statement	Description
const express = require("express")	Imports the Express.js framework and creates the app object used to define middleware and routes.
app.use((req, res, next) => {...})	Registers application-level middleware that runs for every incoming request, before any route handler.
req.method, req.url	Properties of the request object that give the HTTP method (GET, POST, etc.) and the requested URL path.
console.log(...)	Prints the method and URL of the incoming request to the terminal, forming the log entry.
next()	Passes control from the middleware to the next matching function in the pipeline (here, the route handler); without it the request would never reach a route.
app.get("/", ...), app.get("/student", ...), app.get("/about", ...)	Route handlers that run after the logging middleware and send the final response using res.send().
app.listen(3000, ...)	Starts the server on port 3000 and prints a confirmation message once it is ready.

7. TEST CASES AND EXECUTION DATA

Test Case	Request	Expected Console Log	Status
TC-01	GET /	Request Method: GET, Request URL: /	Pass
TC-02	GET /student	Request Method: GET, Request URL: /student	Pass
TC-03	GET /about	Request Method: GET, Request URL: /about	Pass
TC-04	GET /unknown	Request Method: GET, Request URL: /unknown logged, then Express's default "Cannot GET /unknown" is sent (no matching route)	Pass
TC-05	Multiple requests in sequence	A separate log entry is printed for every request, in the order the requests arrive	Pass

8. OUTPUT

Actual output produced by running the program above (server started and a GET request made to the home route "/"), as shown in the browser and the integrated terminal:

Browser window: a page at <http://localhost:3000/> displaying "Welcome to Middleware Demo".

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node middleware.js
Server running on port 3000
Request Method: GET
Request URL: /
```

Fig. 8.1 — Middleware logging the method and URL of the incoming GET / request

9. RESULT

Thus, the concept of middleware in Express.js was studied and its role in the request-response cycle was explained. A logging middleware was successfully developed using `app.use()`, which records the HTTP method and URL of every incoming request to the console and then calls `next()` to pass control to the appropriate route handler. The behaviour was verified through browser testing across multiple routes, matching the expected results shown above.

PART A — Q7

Node.js Modules: Using the os, path and fs Built-in Modules

1. AIM

To explain what Node.js modules are, and to demonstrate, with a working program, the purpose and usage of three core built-in modules — os, path and fs — by retrieving system information, working with file paths, and writing/reading a file.

2. OBJECTIVE

- To understand what a module is in Node.js and how modules are loaded using require().
- To distinguish between built-in (core), local, and third-party modules.
- To use the os module to retrieve operating system and hardware information.
- To use the path module to construct and inspect file paths in a platform-independent way.
- To use the fs module to write to and read from a file synchronously.

3. THEORY

A module in Node.js is a reusable, self-contained block of JavaScript code, typically stored in its own file, that can be loaded into another file using the require() function. Node.js wraps every file in a module wrapper function, which gives each file its own scope so that variables and functions defined in one file do not leak into another unless they are explicitly exported with module.exports. Modules make code organised, reusable and easier to maintain, since related functionality can be grouped together and imported wherever it is needed.

Node.js modules fall into three categories: built-in (core) modules, which ship with Node.js itself and require no installation (e.g. os, path, fs, http); local modules, which are files created within the project and imported using a relative path (e.g. require("./myModule")); and third-party modules, which are installed from npm and imported by package name (e.g. require("express")).

The **os** module provides information about the operating system and the underlying hardware the Node.js process is running on. Commonly used methods include os.platform() (the operating system platform, e.g. "win32", "linux", "darwin"), os.arch() (the CPU architecture, e.g. "x64"), os.totalmem() and os.freemem() (total and free system memory in bytes), and os.cpus() (details of each logical CPU core). It is useful for writing code that needs to adapt to, or report on, the environment it is running in.

The **path** module provides utilities for working with file and directory paths in a way that is consistent across operating systems (Windows uses backslashes, while Linux/macOS use forward slashes). Commonly used methods include path.join() (safely joins path segments into one path), path.basename() (returns the last portion of a path, i.e. the file name), path.dirname() (returns the directory portion of a path), and path.extname() (returns the file's extension). Using path.join() with __dirname (the absolute path of the current file's directory) is the standard way to build reliable, portable file paths.

The **fs** (File System) module allows a Node.js program to interact with the file system — creating, reading, writing, appending to and deleting files and directories. Commonly used synchronous methods include fs.writeFileSync(path, data) (creates a file or overwrites it if it exists), fs.readFileSync(path, 'utf8') (reads the entire file and returns its contents as a string), fs.appendFileSync(path, data) (adds data to the end of an existing file), and fs.existsSync(path) (checks whether a file or directory exists). Asynchronous counterparts (fs.writeFile, fs.readFile, etc.) perform the same operations without blocking the rest of the program, using callbacks or promises.

Together, these three modules show how Node.js's built-in modules let a program inspect its environment (os), build correct file paths regardless of platform (path), and persist data to disk (fs) — all without installing any external package.

4. ALGORITHM / PROCEDURE

1. Create a project folder and a file named app.js.
2. Import the built-in os, path and fs modules using require().
3. Use os.platform(), os.arch() and os.totalmem() to print the operating system, CPU architecture and total memory.

4. Use `path.join(__dirname, "student.txt")` to build the full path of a file inside the project folder.
5. Use `path.basename()`, `path.dirname()` and `path.extname()` on that path to print the file name, directory and extension.
6. Use `fs.writeFileSync()` to create/overwrite `student.txt` with sample text content.
7. Use `fs.readFileSync()` to read the file back and print its content to the console.
8. Save the file and run it using the command: `node app.js`
9. Observe the console output and verify that `student.txt` was created in the project folder with the expected content.

5. PROGRAM

app.js

```
const os = require("os");
const path = require("path");
const fs = require("fs");

// OS Module
console.log("Operating System:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory:", os.totalmem());

// Path Module
const filePath = path.join(__dirname, "student.txt");
console.log("File Path:", filePath);
console.log("File Name:", path.basename(filePath));
console.log("Directory:", path.dirname(filePath));
console.log("Extension:", path.extname(filePath));

// FS Module
fs.writeFileSync(filePath, "Welcome to Node.js");
const data = fs.readFileSync(filePath, "utf8");
console.log("File Content:", data);
Run command: node app.js
```

6. CODE EXPLANATION

Tag / Statement	Description
<code>const os = require("os")</code>	Imports the built-in <code>os</code> module, giving access to operating system and hardware information.
<code>const path = require("path")</code>	Imports the built-in <code>path</code> module, used to build and inspect file paths in a platform-independent way.
<code>const fs = require("fs")</code>	Imports the built-in File System module, used to write to and read from files on disk.
<code>os.platform(), os.arch(), os.totalmem()</code>	Return the operating system platform, the CPU architecture and the total system memory in bytes.
<code>path.join(__dirname, "student.txt")</code>	Builds the absolute path of <code>student.txt</code> inside the current directory, joining segments correctly for the current OS.
<code>path.basename()</code> / <code>path.dirname()</code> / <code>path.extname()</code>	Extract the file name, the containing directory and the file extension from the full path.
<code>fs.writeFileSync(filePath, "Welcome to Node.js")</code>	Creates <code>student.txt</code> (or overwrites it if it already exists) with the given text content.
<code>fs.readFileSync(filePath, "utf8")</code>	Reads the full content of <code>student.txt</code> as a UTF-8 string.
<code>console.log("File Content:", data)</code>	Prints the content that was just read back from the file, confirming the write/read cycle worked.

7. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Status
TC-01	os.platform(), os.arch()	Prints the current OS platform (e.g. win32) and CPU architecture (e.g. x64)	Pass
TC-02	os.totalmem()	Prints total system memory in bytes as a positive number	Pass
TC-03	path.join(__dirname, "student.txt")	Prints the correct absolute path of student.txt inside the project folder	Pass
TC-04	path.basename() / dirname() / extname()	Prints student.txt, the project directory path, and .txt respectively	Pass
TC-05	fs.writeFileSync() then fs.readFileSync()	student.txt is created with the text "Welcome to Node.js" and the same text is read back and printed	Pass
TC-06	Run node app.js a second time	student.txt is overwritten with the same content; output remains identical (no error)	Pass

8. OUTPUT

Actual output produced by running the program above from the command line:

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node app.js
Operating System: win32
CPU Architecture: x64
Total Memory: 16407719936
File Path: C:\Users\aruna\OneDrive\Desktop\fullstack\student.txt
File Name: student.txt
Directory: C:\Users\aruna\OneDrive\Desktop\fullstack
Extension: .txt
File Content: Welcome to Node.js
PS C:\Users\aruna\OneDrive\Desktop\fullstack>
```

Fig. 8.1 — Console output showing OS details, path breakdown and file content written/read using os, path and fs

9. RESULT

Thus, Node.js modules were studied, along with the purpose of the built-in os, path and fs modules. The os module was used to display the operating system, CPU architecture and total memory; the path module was used to build and break down a file path into its name, directory and extension; and the fs module was used to write content to student.txt and read it back successfully. The behaviour was verified through command-line testing, matching the expected results shown above.

PART A — Q8

NPM and package.json in Node.js: Installing and Using an External Package

1. AIM

To explain NPM (Node Package Manager) and the role of package.json in a Node.js application, and to describe and demonstrate the steps involved in installing an external package and using it in a working Node.js program.

2. OBJECTIVE

- To understand what NPM is and why it is used in Node.js development.
- To understand the purpose and structure of the package.json file.
- To learn the steps involved in initialising a Node.js project and installing an external package.
- To demonstrate the use of an installed external package (chalk) inside a Node.js application.

3. THEORY

NPM (Node Package Manager) is the default package manager that is bundled with Node.js. It is used to install, share, update and manage reusable pieces of code, called packages or modules, that other developers have published. Instead of writing every piece of functionality from scratch, a developer can search the NPM registry for a package that already solves the problem and install it into the project with a single command. NPM also manages the versions of these packages and keeps track of which versions a project depends on, so that the same project can be set up identically on any other machine.

package.json is a JSON file kept at the root of every Node.js project. It acts as the manifest, or identity card, of the project. It stores metadata such as the project's name, version, description, entry point (main file), author and license, and, most importantly, it lists the project's dependencies (external packages required to run the project) and devDependencies (packages required only during development, such as testing tools). When a package is installed with the `--save` behaviour (the default in modern NPM), its name and version range are automatically added to the dependencies section of package.json. This allows anyone who receives the project folder to recreate the exact same set of packages simply by running `'npm install'`, without needing to send the actual package files.

Whenever a package is installed, NPM downloads it into a local folder named `node_modules` inside the project, and records the exact resolved version in a companion file called `package-lock.json`, which ensures that every install produces the same dependency versions. The `node_modules` folder and the lock file together with package.json form the complete dependency management system of a Node.js project.

4. STEPS TO INSTALL AND USE AN EXTERNAL PACKAGE

1. Create a project folder and open a terminal inside it.
2. Run `npm init -y` to generate a default package.json file (the `-y` flag accepts all default values automatically).
3. Run `npm install chalk` (or the shorthand `npm i chalk`) to download the 'chalk' package from the NPM registry. NPM creates the `node_modules` folder, downloads the package into it, and adds an entry for 'chalk' under "dependencies" in package.json.
4. Import the package inside the application file using `const chalk = require("chalk");`
5. Use the functions/methods exposed by the package (for example `chalk.blue()`, `chalk.green()`, `chalk.red()`) anywhere in the program to use its functionality.
6. Run the application using `node app.js`; Node.js resolves the `'require("chalk")'` call by looking inside the local `node_modules` folder.

5. PROGRAM

Step 1 — Initialise the project and install the package (terminal):

```
$ npm init -y
$ npm install chalk
```

Step 2 — app.js (Student Management System using the 'chalk' package):

```
const chalk = require("chalk");

// Student details
```

```

const students = [
  { name: "Aruna",    rollNo: 101, branch: "AI & ML", marks: 85 },
  { name: "Priya",    rollNo: 102, branch: "CSE",      marks: 92 },
  { name: "Divya",    rollNo: 103, branch: "ECE",      marks: 78 },
  { name: "Keerthi",  rollNo: 104, branch: "IT",       marks: 88 }
];

// Heading
console.log(chalk.blue("====="));
console.log(chalk.green("      STUDENT MANAGEMENT SYSTEM"));
console.log(chalk.blue("====="));
console.log();

// Display students
console.log(chalk.yellow("Student Details"));
console.log("-----");

students.forEach((student) => {
  console.log(chalk.cyan("Name      : ") + student.name);
  console.log(chalk.cyan("Roll No  : ") + student.rollNo);
  console.log(chalk.cyan("Branch   : ") + student.branch);
  console.log(chalk.cyan("Marks    : ") + student.marks);

  // Grade calculation
  let grade;
  if (student.marks >= 90) grade = "A+";
  else if (student.marks >= 80) grade = "A";
  else if (student.marks >= 70) grade = "B";
  else if (student.marks >= 60) grade = "C";
  else grade = "Fail";

  if (grade === "Fail") {
    console.log(chalk.red("Grade      : " + grade));
  } else {
    console.log(chalk.green("Grade      : " + grade));
  }
  console.log("-----");
});

// Calculate average
let total = 0;
students.forEach((student) => { total = total + student.marks; });
const average = total / students.length;

console.log(chalk.magenta("Total Students : " + students.length));
console.log(chalk.magenta("Total Marks    : " + total));
console.log(chalk.magenta("Average Marks  : " + average.toFixed(2)));

console.log();
console.log(chalk.green("Program executed successfully!"));
console.log(chalk.blue("NPM external package 'chalk' was used."));
Run command: node app.js

```

6. CODE EXPLANATION

Tag / Statement	Description
npm init -y	Generates package.json automatically with default project metadata.
npm install chalk	Downloads the 'chalk' package into node_modules and records it under "dependencies" in package.json.
const chalk = require("chalk");	Imports the installed package so its functions can be used in app.js.
chalk.blue(), chalk.green(), chalk.red(), etc.	Package functions that wrap a string with ANSI colour codes so it prints in colour on the terminal.
students.forEach(...)	Iterates over the array of student objects to print each student's details.
if / else if grade calculation	Assigns a letter grade to each student based on the marks scored.

Tag / Statement	Description
total / average calculation	Accumulates total marks and computes the class average using a <code>forEach</code> loop.

7. OUTPUT

Running the program with `node app.js` produces a colour-formatted console report: a blue-and-green banner, each student's Name, Roll No, Branch and Marks in cyan, a grade in green (or red for a fail), followed by the total students, total marks and average marks in magenta, and a final success message.

Sample output (colours rendered as plain text): Aruna (101, AI & ML, 85, Grade A), Priya (102, CSE, 92, Grade A+), Divya (103, ECE, 78, Grade B), Keerthi (104, IT, 88, Grade A); Total Students: 4, Total Marks: 343, Average Marks: 85.75, followed by "Program executed successfully!" and "NPM external package 'chalk' was used."

8. RESULT

Thus, NPM and `package.json` were studied and explained. NPM was understood as Node.js's package manager used to install and manage external packages, and `package.json` was understood as the project manifest that records project metadata and dependencies. The steps to install an external package (`npm init`, `npm install`, `require`, `use`) were demonstrated by installing the 'chalk' package and using it to build a colour-formatted Student Management System, confirming that an external NPM package can be installed and used successfully in a Node.js application.